

# IMMUTABLE GOLD — Value Stream & Capability Map

## Combined Component Reproducibility Pack v1.0

### Gold Status Declaration

| Field             | Value                                                                                                                                                                     |
|-------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Status            | IMMUTABLE GOLD BASELINE                                                                                                                                                   |
| Version           | v1.0                                                                                                                                                                      |
| File output       | Single <code>.html</code> file — no external CSS or JS files                                                                                                              |
| Reference browser | Microsoft Edge (Chromium) / Chrome                                                                                                                                        |
| Change rule       |  No changes to geometry, layout, interaction semantics, class names, or data structure |
| Allowed changes   |  Additive only — data content, href values on process pills, additional value stages   |

Violating any rule below means the component is **no longer Gold-compliant**.

## 1. Architecture — How the Two Components Connect

This file combines two previously separate Gold components into one unified page:

| Layer | Component                      | Role                                    |
|-------|--------------------------------|-----------------------------------------|
| Top   | Value Stream Chevron Component | Navigation — user selects a Value Stage |
|       |                                | Detail — displays capabilities for the  |

|               |                               |                |
|---------------|-------------------------------|----------------|
| <b>Bottom</b> | L2–L3 Inline Capability Panel | selected stage |
|---------------|-------------------------------|----------------|

**Connection rule:** Clicking a chevron calls `selectStage(index)` . This function reads the `DATA` array at that index, renders the L2/L3 panel into `#panelContent` , and hides the empty state. The two components share a single `DATA` array — there is no separate data source per component.

---

## 2. Non-Negotiable Invariants

### Hierarchy

- Value Stage → L2 → L3
- L3 is the atomic capability unit; no further decomposition in the UI

### Chevron Component

- Value stages are chevrons — not rectangles, not cards
- Layout is left → right only, evenly distributed across full container width
- No horizontal or vertical scrolling
- Text length must not affect layout
- Every chevron has: solid background fill, white border around entire chevron shape (including arrowhead), hover = lighter fill, selected = darker fill
- Only **one** value stage may be selected at a time
- Selection simultaneously highlights: the selected chevron AND the Common Capabilities chevron below
- Hover tooltip text is supplied via the native HTML `title` attribute

### Capability Panel

- L2 capabilities expand and collapse inline on click
- L3 renders only when its parent L2 is open
- The first L2 auto-opens when a new stage is selected
- L3 uses a 3 rows × 4 columns CSS Grid layout

- Column 1 (Description) is CSS-grid-merged across all 3 data rows
- Column 2 = Related Policy (plain text, reference only)
- Column 3 = Related Process (rendered as clickable pill anchor)
- Column 4 = Maturity (colour-coded badge, enumerated values only)
- Process pills are `<a>` anchors; `href="#"` until navigation is explicitly wired

## JavaScript

- Vanilla JS only — no framework, no async logic, no external libraries
- No DOM mutation outside the defined scope
- Selection state is managed by adding/removing CSS classes ( `is-selected` , `is-open` , `is-highlighted` )

---

## 3. CSS Design Tokens ( `:root` variables — copy exactly)

```
:root {
  --chevron-default: #e8eef3;
  --chevron-hover: #d6e4ef;
  --chevron-selected: #4f81bd;
  --chevron-border: #ffffff;
  --common-default: #efe6c8;
  --common-highlight: #e2c873;
  --pill-bg: #e8eef3;
  --pill-border: #4f81bd;
  --pill-text: #4f81bd;
  --text-dark: #1e2d3d;
  --text-mid: #4a5f72;
  --text-light: #7a95aa;
  --surface: #f0f4f8;
  --surface-raised: #ffffff;
  --border-subtle: #dce6ef;
  --accent: #4f81bd;
  --accent-dark: #2d5a8e;
  --l2-header-bg: #2d5a8e;
  --l2-header-text: #ffffff;
  --l3-header-bg: #dce6ef;
  --l3-row-alt: #f5f8fb;
}
```

---

## 4. Typography

Font stack loaded from Google Fonts. Include this `<link>` tag in `<head>` before any `<style>`:

```
<link href="https://fonts.googleapis.com/css2?family=Sora:wght@400;500;600;700&fa
```

| Usage                                                        | Family                      | Weight             |
|--------------------------------------------------------------|-----------------------------|--------------------|
| Body / all UI text                                           | 'Sora', sans-serif          | 400, 500, 600, 700 |
| Eyebrow labels, section labels, panel title, maturity badges | 'JetBrains Mono', monospace | 400, 500           |

body rule:

```
body {  
  font-family: 'Sora', sans-serif;  
  background: var(--surface);  
  color: var(--text-dark);  
  min-height: 100vh;  
}
```

---

## 5. Complete CSS (copy verbatim)

```
/* — RESET — */  
*, *::before, *::after { box-sizing: border-box; margin: 0; padding: 0; }  
  
/* — SHELL — */  
.shell {  
  max-width: 1280px;  
  margin: 0 auto;  
  padding: 40px 32px 80px;  
}  
  
/* — PAGE HEADER — */  
.page-header { margin-bottom: 36px; }  
  
.page-header__eyebrow {
```

```

    font-family: 'JetBrains Mono', monospace;
    font-size: 11px;
    font-weight: 500;
    letter-spacing: 0.14em;
    text-transform: uppercase;
    color: var(--accent);
    margin-bottom: 8px;
}

.page-header__title {
    font-size: 28px;
    font-weight: 700;
    color: var(--text-dark);
    line-height: 1.2;
}

.page-header__sub {
    margin-top: 6px;
    font-size: 14px;
    color: var(--text-mid);
    font-weight: 400;
}

/* — SECTION LABEL ————— */
.section-label {
    font-family: 'JetBrains Mono', monospace;
    font-size: 10px;
    font-weight: 500;
    letter-spacing: 0.12em;
    text-transform: uppercase;
    color: var(--text-light);
    margin-bottom: 10px;
    padding-left: 4px;
}

/* =====
   CHEVRON COMPONENT
   ===== */

.value-stream { margin-bottom: 4px; }

.value-stream__track {
    display: grid;
    grid-auto-flow: column;
    grid-auto-columns: 1fr;
    width: 100%;
}

/* VALUE STAGE CHEVRONS */
.value-stream__chevron {

```

```

    position: relative;
    padding: 3px;
    cursor: pointer;
}
.value-stream__chevron::before {
    content: "";
    position: absolute;
    inset: 0;
    background: var(--chevron-border);
    clip-path: polygon(
        0 0,
        calc(100% - 26px) 0,
        100% 50%,
        calc(100% - 26px) 100%,
        0 100%,
        26px 50%
    );
}
.value-stream__chevron-content {
    position: relative;
    background: var(--chevron-default);
    padding: 16px 30px 16px 26px;
    text-align: center;
    font-size: 13px;
    font-weight: 600;
    line-height: 1.3;
    color: var(--text-dark);
    clip-path: polygon(
        0 0,
        calc(100% - 26px) 0,
        100% 50%,
        calc(100% - 26px) 100%,
        0 100%,
        26px 50%
    );
    transition: background 0.18s ease, color 0.18s ease;
    user-select: none;
}
.value-stream__chevron:hover .value-stream__chevron-content {
    background: var(--chevron-hover);
}
.value-stream__chevron.is-selected .value-stream__chevron-content {
    background: var(--chevron-selected);
    color: #ffffff;
}

```

```

/* COMMON CAPABILITIES */

```

```

.value-stream__common { margin-top: 6px; }

.value-stream__common-chevron {
  position: relative;
  padding: 3px;
}
.value-stream__common-chevron::before {
  content: "";
  position: absolute;
  inset: 0;
  background: var(--chevron-border);
  clip-path: polygon(
    0 0,
    calc(100% - 30px) 0,
    100% 50%,
    calc(100% - 30px) 100%,
    0 100%
  );
}
.value-stream__common-content {
  position: relative;
  background: var(--common-default);
  padding: 11px 44px 11px 20px;
  text-align: center;
  font-size: 13px;
  font-weight: 600;
  color: var(--text-dark);
  clip-path: polygon(
    0 0,
    calc(100% - 30px) 0,
    100% 50%,
    calc(100% - 30px) 100%,
    0 100%
  );
  transition: background 0.18s ease;
}
.value-stream__common.is-highlighted .value-stream__common-content {
  background: var(--common-highlight);
}

/* =====
  CAPABILITY PANEL
  ===== */

.capability-panel {
  margin-top: 32px;
  display: none;
  animation: panelIn 0.25s ease both;

```

```

}
.capability-panel.is-visible { display: block; }

@keyframes panelIn {
  from { opacity: 0; transform: translateY(8px); }
  to   { opacity: 1; transform: translateY(0); }
}

.panel-title {
  font-size: 11px;
  font-family: 'JetBrains Mono', monospace;
  letter-spacing: 0.12em;
  text-transform: uppercase;
  color: var(--text-light);
  margin-bottom: 14px;
  padding-left: 2px;
}

/* — L2 Block ————— */
.l2-block {
  background: var(--surface-raised);
  border: 1px solid var(--border-subtle);
  border-radius: 10px;
  overflow: hidden;
  margin-bottom: 14px;
  box-shadow: 0 2px 8px rgba(79,129,189,0.06);
}

.l2-header {
  display: flex;
  align-items: center;
  gap: 12px;
  padding: 14px 18px;
  background: var(--l2-header-bg);
  cursor: pointer;
  user-select: none;
}

.l2-header__toggle {
  width: 22px;
  height: 22px;
  border-radius: 50%;
  border: 1.5px solid rgba(255,255,255,0.4);
  display: flex;
  align-items: center;
  justify-content: center;
  color: #fff;
  font-size: 14px;

```



```

    font-weight: 700;
    flex-shrink: 0;
    transition: background 0.15s;
}
.l2-header:hover .l2-header__toggle { background: rgba(255,255,255,0.15); }
.l2-header__name {
    font-size: 14px;
    font-weight: 700;
    color: var(--l2-header-text);
    flex: 1;
}
.l2-header__desc {
    font-size: 12px;
    color: rgba(255,255,255,0.72);
    font-weight: 400;
    max-width: 520px;
    line-height: 1.4;
}

.l2-body { display: none; padding: 16px; }
.l2-block.is-open .l2-body { display: block; }

/* — L3 Block ————— */
.l3-block {
    border: 1px solid var(--border-subtle);
    border-radius: 7px;
    overflow: hidden;
    margin-bottom: 10px;
}
.l3-block:last-child { margin-bottom: 0; }

.l3-name-bar {
    background: #eaf0f7;
    padding: 9px 14px;
    font-size: 12px;
    font-weight: 700;
    color: var(--accent-dark);
    letter-spacing: 0.02em;
    border-bottom: 1px solid var(--border-subtle);
}

/* — L3 Grid ————— */
.l3-grid {
    display: grid;
    grid-template-columns: 1fr 160px 180px 110px;
    font-size: 12px;
}

```

```

.l3-head {
  background: var(--l3-header-bg);
  padding: 8px 12px;
  font-weight: 700;
  font-size: 11px;
  letter-spacing: 0.05em;
  text-transform: uppercase;
  color: var(--accent-dark);
  border-bottom: 1px solid var(--border-subtle);
}

.l3-desc-merged {
  grid-column: 1;
  grid-row: 2 / span 3;
  padding: 12px 14px;
  color: var(--text-mid);
  line-height: 1.55;
  font-size: 12px;
  border-right: 1px solid var(--border-subtle);
  display: flex;
  align-items: flex-start;
}

.l3-cell {
  padding: 10px 12px;
  color: var(--text-mid);
  border-bottom: 1px solid var(--border-subtle);
  display: flex;
  align-items: center;
}

.l3-cell:last-child,
.l3-grid > .l3-cell:nth-last-child(-n+3) { border-bottom: none; }

.l3-row-alt { background: var(--l3-row-alt); }

/* Process pill CTA */
a.process-pill {
  display: inline-block;
  padding: 4px 10px;
  border-radius: 999px;
  background: var(--pill-bg);
  border: 1px solid var(--pill-border);
  color: var(--pill-text);
  font-weight: 600;
  font-size: 11px;
  text-decoration: none;
}

```

```

    font-family: 'Sora', sans-serif;
    transition: background 0.15s, color 0.15s;
    white-space: nowrap;
}
a.process-pill:hover { background: var(--pill-border); color: #ffffff; }

/* Maturity badge */
.maturity-badge {
    display: inline-block;
    font-size: 10px;
    font-family: 'JetBrains Mono', monospace;
    font-weight: 500;
    letter-spacing: 0.06em;
    padding: 3px 8px;
    border-radius: 4px;
    text-transform: uppercase;
}

.maturity-badge--defined { background: #dff0e0; color: #2a7a3b; }
.maturity-badge--managed { background: #fff5d6; color: #8a6500; }
.maturity-badge--planned { background: #eae6f4; color: #5a3fa0; }
.maturity-badge--initial { background: #fde8e8; color: #b53030; }
.maturity-badge--optimised { background: #d6f0f5; color: #1a6e7a; }

/* Empty state */
.empty-state {
    margin-top: 40px;
    text-align: center;
    color: var(--text-light);
    font-size: 14px;
    padding: 48px 24px;
    border: 2px dashed var(--border-subtle);
    border-radius: 12px;
}

.empty-state__icon { font-size: 32px; margin-bottom: 10px; }

```

---

## 6. Complete HTML Structure (copy verbatim)

```

<!DOCTYPE html>
<html lang="en">
<head>
<meta charset="UTF-8" />
<meta name="viewport" content="width=device-width, initial-scale=1.0" />
<title>Value Stream - Capability Architecture</title>
<link href="https://fonts.googleapis.com/css2?family=Sora:wght@400;500;600;700&fa

```

```

<style>
  /* [INSERT ALL CSS FROM SECTION 5 HERE – including :root block from Section 3]
</style>
</head>
<body>
<div class="shell">

  <!-- PAGE HEADER -->
  <header class="page-header">
    <p class="page-header__eyebrow">SRO Transformation Architecture</p>
    <h1 class="page-header__title">Value Stream & Capability Map</h1>
    <p class="page-header__sub">Select a value stage to explore its L2 capabilities</p>
  </header>

  <!-- CHEVRON COMPONENT -->
  <p class="section-label">Value Stages</p>
  <div class="value-stream" id="valueStream">

    <div class="value-stream__track" id="chevronTrack">
      <!-- Chevrons injected by JS – do not add chevrons here manually -->
    </div>

    <div class="value-stream__common" id="commonCapabilities">
      <div class="value-stream__common-chevron">
        <div class="value-stream__common-content"
          title="Capabilities shared across all value stages">
            Common Capabilities
        </div>
      </div>
    </div>

  </div>

  <!-- CAPABILITY PANEL – hidden until a stage is selected -->
  <div class="capability-panel" id="capabilityPanel">
    <p class="panel-title" id="panelTitle"></p>
    <div id="panelContent"></div>
  </div>

  <!-- EMPTY STATE – visible on load, hidden after first selection -->
  <div class="empty-state" id="emptyState">
    <div class="empty-state__icon">👉</div>
    Select a value stage above to explore its capabilities.
  </div>

</div>
<script>

```

```
/* [INSERT ALL JAVASCRIPT FROM SECTION 7 HERE] */
</script>
</body>
</html>
```

---

## 7. Complete JavaScript (copy verbatim)

### 7a. Data Model

Every value stage entry must conform to this exact shape. Do not change key names.

```
const DATA = [
  {
    stage: "Assess & Issue Land Tax",
    description: "Core assessment lifecycle from valuation through to final notice",
    rows: [
      {
        name: "Assessment Management",
        description: "Calculate, manage, and finalise land tax assessments in accordance with the Land Tax Act 2005",
        rows: [
          {
            name: "Liability Calculation",
            description: "Determine tax liability across primary, exceptional, and transitional land",
            rows: [
              { policy: "Land Tax Act 2005", process: "Calculate Liability", m },
              { policy: "Adjustment Policy", process: "Apply Adjustments", m },
              { policy: "TBD", process: "Automated Calculation", m }
            ]
          },
        ]
      },
      {
        name: "Assessment Finalisation",
        description: "Review, approve, and lock assessments prior to notice generation",
        rows: [
          { policy: "Review Framework", process: "Review Assessment", m },
          { policy: "Approval Policy", process: "Approve Assessment", m },
          { policy: "Lock-down Policy", process: "Finalise Assessment", m }
        ]
      }
    ]
  },
  {
    name: "Notice Generation",
    description: "Produce, format, and dispatch land tax assessment notices to landowners",
    rows: [

```

```

{
  name: "Notice Production",
  description: "Generate notices from finalised assessments and prepare
rows: [
  { policy: "Notice Standards", process: "Generate Notice", m
  { policy: "Template Policy", process: "Format Notice", m
  { policy: "Delivery SLA", process: "Dispatch Notice", m
]
}
]
}
]
},
{
  stage: "Objections & Reviews",
  description: "Manage taxpayer objections and formal review requests against a
12: [
  {
    name: "Objection Lodgement",
    description: "Receive, validate, and register objections against issued a
13: [
    {
      name: "Objection Intake",
      description: "Capture objection grounds and supporting evidence from
rows: [
      { policy: "Objection Policy", process: "Receive Objection", m
      { policy: "Evidence Standards", process: "Validate Evidence", m
      { policy: "Registration SOP", process: "Register Objection", m
    ]
  }
]
},
{
  name: "Review & Determination",
  description: "Assess the merits of objections and issue formal determinat
13: [
  {
    name: "Case Assessment",
    description: "Evaluate objection merits against policy, legislation,
rows: [
    { policy: "Merits Review Policy", process: "Assess Case",
    { policy: "Delegation Framework", process: "Escalate if Required",
    { policy: "Determination SOP", process: "Issue Determination",
  ]
}
]
}
}

```

```

    ]
  },
  {
    stage: "Debt & Collections",
    description: "Identify, pursue, and resolve outstanding land tax debt obligat
12: [
  {
    name: "Debt Identification",
    description: "Identify outstanding amounts and classify debt by risk and
13: [
  {
    name: "Debt Triage",
    description: "Segment debt portfolio by risk profile for prioritised
    rows: [
      { policy: "Debt Policy",          process: "Identify Debt",          m
      { policy: "Risk Matrix",          process: "Classify Risk",          m
      { policy: "Prioritisation SOP", process: "Prioritise Debt",          m
    ]
  }
]
},
{
  name: "Recovery Management",
  description: "Execute recovery strategies including payment plans and enf
13: [
  {
    name: "Payment Arrangement",
    description: "Negotiate and administer payment plans for compliant de
    rows: [
      { policy: "Payment Plan Policy", process: "Offer Arrangement", m
      { policy: "Hardship Guidelines", process: "Assess Hardship", m
      { policy: "Monitoring SOP",      process: "Monitor Compliance", m
    ]
  }
]
}
]
},
{
  stage: "Customer Management",
  description: "Manage taxpayer identity, correspondence, and relationship touc
12: [
  {
    name: "Taxpayer Registration",
    description: "Create and maintain accurate taxpayer identity records.",
    13: [
      {

```

```

        name: "Identity Verification",
        description: "Confirm taxpayer identity against authoritative data so
rows: [
    { policy: "Identity Policy", process: "Verify Identity", m
    { policy: "DVS Standards", process: "Check DVS", m
    { policy: "Exception SOP", process: "Manual Verification", m
    ]
    }
    ]
    }
    ]
    },
    {
        stage: "Reporting & Analytics",
        description: "Produce operational, statutory, and analytical outputs across t
12: [
    {
        name: "Operational Reporting",
        description: "Deliver real-time and scheduled reports on assessment and c
13: [
    {
        name: "Dashboard Production",
        description: "Build and maintain executive and operational dashboards
rows: [
    { policy: "Reporting Policy", process: "Produce Dashboard", m
    { policy: "KPI Framework", process: "Calculate KPIs", m
    { policy: "Distribution SOP", process: "Distribute Reports", m
    ]
    }
    ]
    }
    ]
    }
    ]
    }
    ];

```

## 7b. Helper Function

```

function maturityClass(m) {
    const map = {
        Defined: "defined",
        Managed: "managed",
        Planned: "planned",
        Initial: "initial",
        Optimised: "optimised"
    };
    return map[m] || "defined";
}

```



```
}
```

## 7c. Build Chevrons (runs on page load)

```
const track = document.getElementById("chevronTrack");
const common = document.getElementById("commonCapabilities");

DATA.forEach((stageData, i) => {
  const chev = document.createElement("div");
  chev.className = "value-stream__chevron";
  chev.title = stageData.description;
  chev.dataset.index = i;
  chev.innerHTML = `<div class="value-stream__chevron-content">${stageData.stage}
  chev.addEventListener("click", () => selectStage(i));
  track.appendChild(chev);
});
```

## 7d. Render Functions

```
function renderL3(l3) {
  let html = `<div class="l3-block">
    <div class="l3-name-bar">${l3.name}</div>
    <div class="l3-grid">
      <div class="l3-head">Description</div>
      <div class="l3-head">Related Policy</div>
      <div class="l3-head">Related Process</div>
      <div class="l3-head">Maturity</div>
      <div class="l3-desc-merged">${l3.description}</div>`;

  l3.rows.forEach((row, ri) => {
    const alt = ri % 2 === 1 ? " l3-row-alt" : "";
    html += `
      <div class="l3-cell${alt}">${row.policy}</div>
      <div class="l3-cell${alt}"><a class="process-pill" href="#">${row.process}<
      <div class="l3-cell${alt}"><span class="maturity-badg maturity-badg--${ma
    });

    html += `</div></div>`;
    return html;
  }

function renderL2(l2Data) {
  const l3Html = l2Data.l3.map(renderL3).join("");
  return `
```

```

    <div class="l2-block">
      <div class="l2-header">
        <div class="l2-header__toggle">+</div>
        <div class="l2-header__name">${l2Data.name}</div>
        <div class="l2-header__desc">${l2Data.description}</div>
      </div>
      <div class="l2-body">${l3Html}</div>
    </div>`;
  }

```

## 7e. Selection Logic

```

let currentIndex = -1;

function selectStage(index) {
  // Deselect all chevrons
  document.querySelectorAll(".value-stream__chevron")
    .forEach(c => c.classList.remove("is-selected"));

  // Select the clicked chevron
  const chevrons = document.querySelectorAll(".value-stream__chevron");
  chevrons[index].classList.add("is-selected");

  // Highlight common capabilities
  common.classList.add("is-highlighted");

  // Resolve DOM targets
  const stageData = DATA[index];
  const panel      = document.getElementById("capabilityPanel");
  const title      = document.getElementById("panelTitle");
  const content    = document.getElementById("panelContent");
  const empty      = document.getElementById("emptyState");

  // Render panel content
  title.textContent = `Capabilities - ${stageData.stage}`;
  content.innerHTML = stageData.l2.map(renderL2).join("");

  // Hide empty state, show panel
  empty.style.display = "none";
  panel.classList.add("is-visible");

  // Wire L2 expand/collapse for newly rendered blocks
  content.querySelectorAll(".l2-header").forEach(header => {
    header.addEventListener("click", () => {
      const block = header.closest(".l2-block");
      const toggle = header.querySelector(".l2-header__toggle");
    });
  });
}

```

```

        const isOpen = block.classList.toggle("is-open");
        toggle.textContent = isOpen ? "-" : "+";
    });
});

// Auto-open first L2
const firstBlock = content.querySelector(".l2-block");
const firstToggle = firstBlock.querySelector(".l2-header__toggle");
firstBlock.classList.add("is-open");
firstToggle.textContent = "-";

currentIndex = index;
}

```

## 8. Maturity Values — Enumeration

Only these five values are permitted in the `maturity` field of any `rows` entry:

| Value     | Badge colour (bg) | Badge colour (text) | CSS modifier              |
|-----------|-------------------|---------------------|---------------------------|
| Defined   | #dff0e0           | #2a7a3b             | maturity-badge--defined   |
| Managed   | #fff5d6           | #8a6500             | maturity-badge--managed   |
| Planned   | #eae6f4           | #5a3fa0             | maturity-badge--planned   |
| Initial   | #fde8e8           | #b53030             | maturity-badge--initial   |
| Optimised | #d6f0f5           | #1a6e7a             | maturity-badge--optimised |

## 9. DOM ID Reference (do not rename)

| ID           | Element | Purpose                                                 |
|--------------|---------|---------------------------------------------------------|
| valueStream  | <div>   | Outer chevron component wrapper                         |
| chevronTrack | <div>   | Grid container — chevrons injected here by JS           |
|              |         | Common Capabilities chevron — receives <code>is-</code> |

|                    |       |                                                                        |
|--------------------|-------|------------------------------------------------------------------------|
| commonCapabilities | <div> | highlighted <b>class</b>                                               |
| capabilityPanel    | <div> | <b>Panel wrapper — receives is-visible class to show</b>               |
| panelTitle         | <p>   | <b>Text set to</b> Capabilities — {stage name}                         |
| panelContent       | <div> | <b>innerHTML replaced with rendered L2/L3 blocks on each selection</b> |
| emptyState         | <div> | <b>Hidden ( display:none ) after first stage selection</b>             |

## 10. CSS Class State Rules

| Class          | Applied to             | Trigger                                      | Effect                                                  |
|----------------|------------------------|----------------------------------------------|---------------------------------------------------------|
| is-selected    | .value-stream__chevron | Chevron click                                | Background → --chevron-selected (#4f81bd), text → white |
| is-highlighted | .value-stream__common  | Any chevron click (permanent once triggered) | Background → --common-highlight (#e2c873)               |
| is-visible     | .capability-panel      | Any chevron click                            | display: none → display: block with panelIn animation   |
| is-open        | .l2-block              | L2 header click                              | .l2-body becomes visible; toggle symbol changes + → -   |

## 11. Copilot Reproduction Prompt (paste this exactly)

Reproduce the IMMUTABLE GOLD Value Stream & Capability Map component exactly as d

Rules:

- Output a single self-contained .html file with no external CSS or JS files
- Include the Google Fonts <link> for Sora and JetBrains Mono exactly as specifie
- Copy all CSS verbatim including :root design tokens — do not alter any value

- Copy all HTML structure verbatim – do not rename any ID or class
  - Copy all JavaScript verbatim – do not alter selection logic, render functions,
  - Populate the DATA array with the exact 5 value stages, L2s, L3s, and rows speci
  - Do not add frameworks, libraries, or async logic
  - Do not change geometry, clip-paths, grid columns, or animation keyframes
  - Maturity values must only use the five permitted enumerated values
  - Process pills are <a> anchors with href="#" – do not wire navigation unless exp
  - The first L2 auto-opens on stage selection – preserve this behaviour exactly
- 

**Owner:** Mark Gittins **Status:** Locked for reuse across SRO transformation artefacts  
**Derived from:** Value Stream Chevron Component Gold v1.3 + L2–L3 Inline Capability  
Pattern Gold v1.3